

ENRICO FIORI FERRAZ

POSTECH

IA PARA DEVS

FINE-TUNING E RAG PARA DOCUMENTOS

AULA 01

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?.....	9
REFERÊNCIAS.....	10

EMSE

O QUE VEM POR AÍ?

Já parou para pensar que os grandes modelos de linguagem conhecidos como LLMs (Large Language Models) são treinados com uma quantidade imensa de dados? O GPT-4 da OpenAI, por exemplo, foi treinado com praticamente todo o conteúdo da internet aberta. Mas não basta coletar os dados brutos, é preciso também realizar a limpeza e preparação para que o conteúdo esteja no formato de dados exigido pelo modelo escolhido.

Nesta aula, vamos abordar o tópico de pré-processamento de dados brutos, para formatá-los de maneira que o modelo escolhido possa compreender e aprender a partir do dataset.

HANDS ON

Então, vamos à prática! Nosso objetivo nesta aula será extrair os conteúdos dos artigos de notícias [deste site](#) e tratar esses dados para construir o dataset e para que o nosso modelo aprenda com base neles. Mas, para facilitar o processo, também vamos utilizar um dataset complementar, com mais de 10 mil artigos de diversas fontes.

Primeiramente, é necessário configurar nosso ambiente de desenvolvimento. Para essa disciplina, vamos usar o Google Colab, uma ferramenta do Google baseada em Jupyter Notebooks, onde é possível utilizar diferentes hardwares, como CPU e GPU, para impulsionar nossa eficiência e tempo de processamento computacional. Se você nunca usou o Google Colab, acesse o [link](#) e crie sua conta. Caso deseje ainda mais processamento, o Colab oferece um plano “Pro”, que disponibiliza hardwares mais avançados e que diminuem o tempo de execução.

Recomendo que já realize o download do repositório da disciplina [aqui](#), bem como do [dataset complementar](#) e estude sua estrutura.

Com o Colab configurado, crie um arquivo e execute os seguintes comandos para instalar as dependências que vamos utilizar nessa aula. As demais bibliotecas utilizadas já são built-in da linguagem Python.

```
!pip install beautifulsoup4
!pip install openai
!pip install requests
```

Código-fonte 1 – Comandos para instalação de dependências
Fonte: elaborado pelo autor (2024)

Pronto! Seu ambiente já contém tudo que será preciso para esta aula. Agora, assista aos vídeos dessa aula para executar os scripts disponibilizados e vamos extrair e estruturar nossos dados para preparar o dataset!

SAIBA MAIS

Durante os vídeos, você notou que criamos um web scraper para extrair os links contidos na página principal do site, mas o que é e como funciona esse tipo de script? Quando pensamos em extrair dados da web, podemos fazer isso de forma manual, encontrando no código HTML da página o elemento que contém o conteúdo desejado. Mas, para o nosso projeto, será um div da classe “article__content”, conforme a figura abaixo:

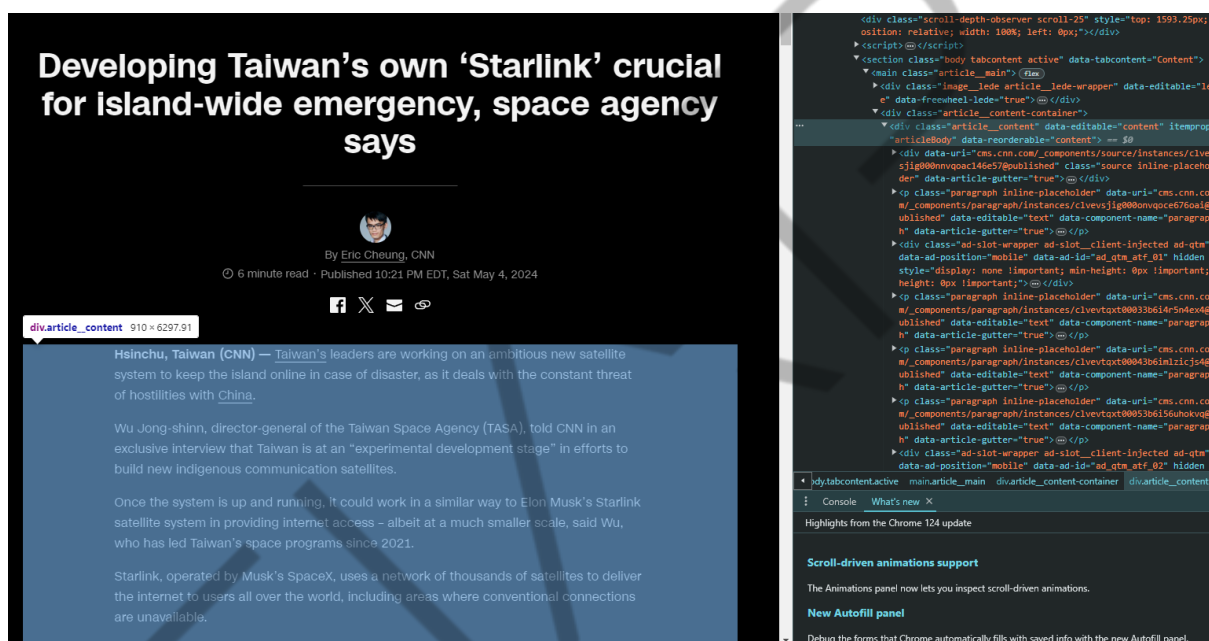


Figura 1 - Análise do código HTML de uma notícia

Fonte: elaborado pelo autor (2024)

Dependendo da quantidade de notícias, fica inviável extrair esse conteúdo de forma manual. São nesses casos que o web scraping proporciona ganhos reais de tempo e produtividade.

O que é Web Scraping?

O web scraping é uma técnica usada para extrair dados de websites. Essa abordagem permite que pessoas desenvolvedoras e analistas de dados coletem dados estruturados de páginas da web de forma automatizada. Esse processo geralmente envolve fazer requisições HTTP para um site específico, analisar o HTML

recebido e extrair as informações necessárias, geralmente utilizando bibliotecas como “Beautiful Soup” ou “Scrapy” em Python.

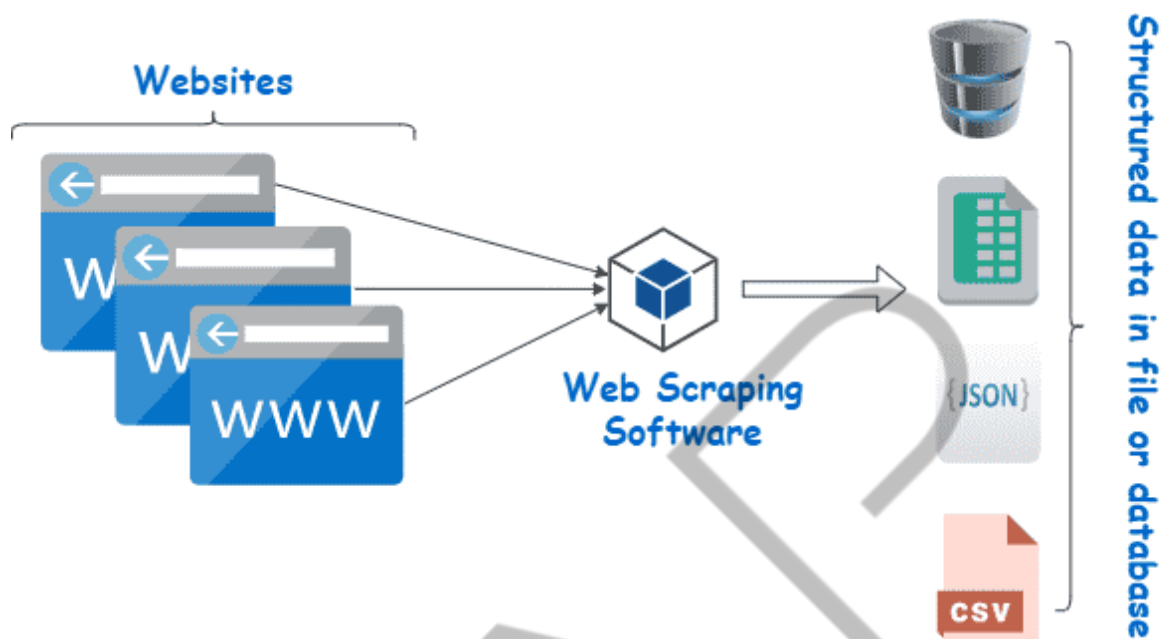


Figura 2 - Análise do código HTML de uma notícia
Fonte: [Kinsta](#) (2024)

O uso de web scraping é particularmente útil em situações nas quais os dados necessários estão disponíveis publicamente, mas não por meio de uma API ou em formato facilmente exportável. Isso inclui casos como análise competitiva, monitoramento de preços, pesquisa de mercado e agregação de notícias.

No entanto, é importante salientar que esse processo também levanta questões éticas e legais. É importante considerar as políticas de uso do site e a legislação local sobre acesso a dados digitais. Em muitos casos, sites possuem termos de serviço que proíbem explicitamente o scraping, especialmente se isso causar uma carga excessiva no servidor ou se envolver a captura de dados protegidos por direitos autorais.

A eficácia do web scraping pode ser impactada por técnicas que os sites usam para bloquear scrapers, como CAPTCHAs, exigência de login ou mudanças frequentes na estrutura do HTML. Isso requer que pessoas desenvolvedoras atualizem seus scripts regularmente para adaptar-se a novas estruturas ou defesas.

No Python, temos algumas bibliotecas muito utilizadas na extração de dados da web, principalmente [Beautiful Soup](#), [Selenium](#) e [Scrapy](#). Caso tenha interesse, leia

a documentação e entenda qual melhor se encaixa no problema com o qual você está lidando.

Apesar do web scraping ser um método de muita utilidade quando preparamos nosso dataset, não são todas as páginas da web que são habilitadas ou que permitem o uso dessas técnicas para extração de dados. Por isso, sempre se certifique que o site é aberto ou obtenha a permissão do titular do site para executar esse procedimento.

Com essas informações, retorne aos códigos e tente criar um web scrapper em algum site que possui um conteúdo que seria útil para você, extraia os dados e realize a limpeza desses dados. Caso queira um desafio maior, inclua uma função que seja capaz de autenticar em CAPTCHAs.

Para aprofundar seus conhecimentos nas técnicas de web scraping, recomendo fortemente a leitura do livro **“Web Scraping with Python”**, de Ryan Mitchell.

Gerando outputs com IA para treinamento do seu modelo

Se você estranhou que utilizamos uma IA para gerar os resumos das notícias que coletamos, na realidade, esse processo é amplamente utilizado para reduzir custos e o tempo de preparação dos dados de treino. Assim como no web scraper, em que automatizamos o processo de obtenção dos dados, a ideia é automatizar o processo de geração do retorno esperado de cada conteúdo que coletamos. Imagine que temos uma base de 1000 artigos de notícias e precisamos treinar um modelo que resuma esses dados. Para nosso treino, precisamos não somente do texto completo, mas também indicar ao modelo qual é o retorno esperado, para que seja treinado de forma eficiente.

Antes dos modelos GPT, essa tarefa era realizada por pessoas que faziam esse processo manualmente e, em muitos casos, como o reconhecimento de objetos em imagens mostrado na figura abaixo:

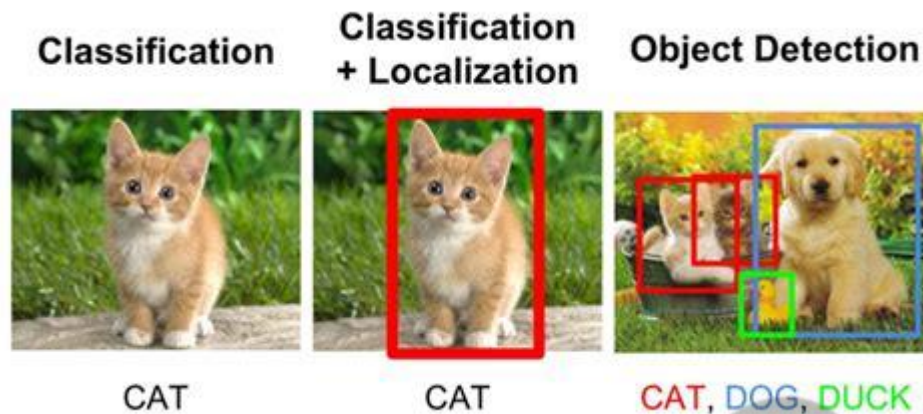


Figura 3 - Reconhecimento de objetos em imagens
 Fonte: [Acervo Lima](#) (2024)

A partir dos modelos de geração de texto, como o gpt-3.5-turbo da OpenAI, já é possível gerar retornos a partir do texto “inputado” pelo usuário. Isso permite preparar nosso dataset sem precisar criar resumos manualmente, aumentando consideravelmente não somente a velocidade de preparação do dataset, mas viabilizando um ganho de escala e da qualidade dos dados produzidos.

Utilize o script **generate-output-for-news.ipynb**, altere o modelo e os parâmetros como “temperature” e depois compare os resultados. Esse exercício auxilia no entendimento de como funcionam esses modelos em tarefas de processamento de linguagem natural (NLP).

Para saber mais, leia o artigo da OpenAI sobre o impacto de LLMs no mercado de trabalho [aqui](#).

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos o que é web scraping e como utilizar essa técnica para obter dados valiosos para o treinamento de nosso modelo, que será realizado na próxima aula. Passamos pelas principais bibliotecas disponíveis para executar na prática a coleta e tratamento dos dados a partir do site de notícias. Nota-se que conseguimos muitos links a partir de apenas uma única página do portal.

Configuramos nosso ambiente no Colab, instalamos as dependências necessárias e, com base nos códigos disponibilizados, utilizamos um dataset externo com uma gama maior de notícias.

Por fim, utilizamos os modelos avançados da OpenAI para gerar os outputs esperados, economizando tempo e aumentando a eficiência e escala do nosso projeto. Com isso, incluímos as informações obtidas do scraper, além de deixar o modelo no formato que ele aprenderá, levando em conta todos esses dados.

Fique ligado, pois, na próxima aula, vamos usar os dados coletados e tratados nesta aula para treinar um foundation model usando fine-tuning para gerar resumos de notícias. A gente se vê lá!

REFERÊNCIAS

MITCHELL, R. **Web Scraping with Python: Collection More Data From The Modern Web**. Sebastopol: O'Reilly Media, 2018.

OPENAI. **GPTs are GPTs: An Early Look at the Labor Market Impact Potential of Large Language Models**. 2023. Disponível em: < <https://arxiv.org/pdf/2303.10130>>. Acesso em: 27 maio 2024.

O QUE é Web Scraping? Como Extrair Legalmente o Conteúdo da Web. KINSTA. 2023. Disponível em: <<https://kinsta.com/pt/base-de-conhecimento/o-que-e-web-scraping/>>. Acesso em: 27 maio 2024.

PALAVRAS-CHAVE

Web Scraping. OpenAI. Coleta e Tratamento de Dados.

EMENDAS



POSTECH